TEST_FILE.CPP

```
//============================================================================
//                      wapwap
//============================================================================
// by      : INSANE
// created : 31/10/2026
//
// purpose : generate pdf with output for single file programs
//----------------------------------------------------------------------------
#include <cmath>
#include <stdio.h>
#include <vector>
#include <cassert>
#include <cstring>

// libharu
#include "../lib/libharu/include/hpdf.h"

#include "config.h"
#include "util/log_io.h"



#define OK (0)
#define ER (1)

// standard colors...
#define BLACK      ((vec3){0.0, 0.0, 0.0})
#define WHITE      ((vec3){1.0, 1.0, 1.0})
#define LIGHT_GRAY ((vec3){0.95, 0.95, 0.95})
#define MID_GRAY   ((vec3){0.9, 0.9, 0.9})
#define RED        ((vec3){1.0, 0.0, 0.0})
#define BLUE       ((vec3){0.0, 0.0, 1.0})
#define GREEN      ((vec3){0.0, 1.0, 0.0})


#define COMMENT_STR "//"
#define TAB_SIZE (4)
char EMPTY_LINE[] = "";


#define min(x, y) ((x) < (y) ? (x) : (y))


// page margin
const float MARGIN_TOP    = 10.0;
const float MARGIN_BOTTOM = 10.0;
const float MARGIN_LEFT   = 10.0;
const float MARGIN_RIGHT  = 10.0;


// cmd line -> files
// cmd line -> compiler path

const char* files[] = {
    "practical_001.cpp"
};
const size_t file_count = sizeof(files) / sizeof(files[0]);

static int ww_add_from_file(struct config *c, const char *f);
static int ww_pushback_a4page(struct config *c);
static int ww_add_heading(struct config *c, const char *h);
static int ww_add_exec_file(struct config *c, const char *f);
static int ww_addlinebg(HPDF_Page page, struct vec3 clr, float x, float y, float width, float height);
```

```
static int ww_textout(struct config *c, const char *msg);
static int ww_copy_page_stats(HPDF_Page from, HPDF_Page to);

static int ww_split_line(HPDF_Page page, char *line, std::vector<char*>* lines_out);
static int ww_is_comment(const char *c);
static float ww_get_curfont_height(HPDF_Page page);
static HPDF_Page ww_get_active_page(struct config *c);

void err_handler(HPDF_STATUS error_no, HPDF_STATUS detail_no, void *user_data);
HPDF_Font init_font(HPDF_Doc pdf, const char *font_file);
HPDF_Page add_a4_page(HPDF_Doc* pdf);
void sanitize_strline_inplace(char *str);


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
int main(int argc, char **argv)
{
    HPDF_Doc pdf = HPDF_New(err_handler, NULL);
    if (!pdf) {
        FAIL_LOG("Failed to create new file");
        return 1;
    }
    WIN_LOG("doc initialized");

    HPDF_Font font_jbm = init_font(pdf, "JetBrainsMono.ttf");
    if (!font_jbm) {
        FAIL_LOG("Failed to load font");
        return 1;
    }
    WIN_LOG("Fonts initialized");

    config c = {0};
    c.pdf = pdf;
    c.book.clear();
    c.it = 0.0;
    c.line_num = true;

    c.font_codetext    = font_jbm;
    c.font_codecomment = font_jbm;
    c.font_heading     = font_jbm;
    c.font_output      = font_jbm;
    c.font_linenum     = font_jbm;

    c.clr_codetext     = BLACK;
    c.clr_codecomment  = BLACK;
    c.clr_output       = BLACK;
    c.clr_heading      = BLACK;
    c.clr_headingdecor = BLACK;
    c.clr_codeline_a   = LIGHT_GRAY;
    c.clr_codeline_b   = MID_GRAY;
    c.clr_linenum      = BLACK;

    ww_pushback_a4page(&c);
    c.it = HPDF_Page_GetHeight(c.book.back()) - MARGIN_TOP;

    for (size_t i = 0; i < file_count; ++i) {
        const char *f = files[i];

        ww_add_heading(&c, f);
        ww_textout(&c, " "); // empty line

        int status_prog = ww_add_from_file(&c, f);
        if (status_prog != OK) {
```

```
            FAIL_LOG("Failed to generate pdf for file : %s", files[i]);
            return 1;
        }

        ww_textout(&c, " "); // empty line
        ww_add_heading(&c, "Output");
        ww_textout(&c, " "); // empty line

        int status_output = ww_add_exec_file(&c, f);
        if (status_output != OK) {
            FAIL_LOG("Failed to generate pdf for file : %s", files[i]);
            return 1;
        }


        WIN_LOG("%s generated %.1f%%", files[i], 100.0 * (float)i / (float)file_count);
    }


    // Write file and exit
    HPDF_SaveToFile(pdf, "output.pdf");
    HPDF_Free(pdf);
    return 0;
}


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_add_from_file(struct config *c, const char *f)
{
    FILE *file = fopen(f, "r");
    if (!file) {
        FAIL_LOG("Couldn't open file %s", f);
        return ER;
    }

    float pwidth = HPDF_Page_GetWidth(c->book.back());
    float pheight = HPDF_Page_GetHeight(c->book.back());

    HPDF_Page start_page = ww_get_active_page(c);
    HPDF_Page_SetFontAndSize(start_page, c->font_codetext, 8.0); // use default code font to calculate line wrappin
g.
    float code_font_height = ww_get_curfont_height(start_page);
    float code_font_descend = abs(HPDF_Page_GetCurrentFontSize(start_page) * HPDF_Font_GetDescent(HPDF_Page_GetCurr
entFont(start_page)) / 1000.0);

    std::vector<char*> lines;
    std::vector<char*> split_lines;
    char *line = NULL; size_t linelen = 0;
    while (getline(&line, &linelen, file) > 0) {
        split_lines.clear();

        // count all tab chars
        size_t tab_count = 0;
        for (size_t i = 0; i < linelen; ++i)
            if (line[i] == '\t')
                ++tab_count;

        // Allocate new buffer to compensate for expanded tabs
        char *new_buf = (char*)calloc(linelen + tab_count * (TAB_SIZE - 1) + 10, 1);
        strncpy(new_buf, line, linelen);

        // expand all tabs
        sanitize_strline_inplace(new_buf);
```

```
        // Split the line for the sake of line-wrapping.
        ww_split_line(start_page, new_buf, &split_lines);
        if (split_lines.empty()) {
            lines.push_back(EMPTY_LINE);
            LOG("Empty line");
        }
        else {
            for (char *line_piece : split_lines)
                lines.push_back(line_piece);
            LOG("Split line into %zu pieces", split_lines.size());
        }

        free(new_buf);
    }

    int total_boxes = lines.size();
    int boxes_left = lines.size();
    HPDF_Page last_boxed_page = NULL;
    for (size_t i = 0; i < lines.size(); ++i) {
        HPDF_Page active_page = ww_get_active_page(c);

        if (boxes_left > 0 && last_boxed_page != active_page) {
            float box_it = c->it - code_font_descend; // Acknowledge the subtracted font descend
            assert(box_it > MARGIN_BOTTOM && box_it < pheight - MARGIN_TOP && "Invalid box iterator");

            int max_boxes = (box_it - MARGIN_BOTTOM) / code_font_height;
            int drawable_boxes = min(max_boxes, boxes_left);
            float merged_box_height = (float)drawable_boxes * code_font_height;
            ww_addlinebg(active_page, c->clr_codeline_a, MARGIN_LEFT, box_it - merged_box_height, pwidth - MARGIN_L
EFT - MARGIN_RIGHT, merged_box_height);

            // Draw alternate boxes
            for (int box_i = 0; box_i < drawable_boxes / 2; ++box_i) {
                ww_addlinebg(active_page,
                        c->clr_codeline_b,
                        MARGIN_LEFT,
                        box_it - (float)(box_i + 1) * code_font_height * 2.0,
                        pwidth - MARGIN_LEFT - MARGIN_RIGHT,
                        code_font_height);

            }

            // Draw border lines over the boxes
            HPDF_Page_SetLineWidth(active_page, 1.0);
            HPDF_Page_SetRGBStroke(active_page, 0.0, 0.0, 0.0);
            HPDF_Page_MoveTo(active_page, MARGIN_LEFT, box_it);
            HPDF_Page_LineTo(active_page, MARGIN_LEFT, box_it - merged_box_height);
            HPDF_Page_Stroke(active_page);
            HPDF_Page_MoveTo(active_page, pwidth - MARGIN_RIGHT, box_it);
            HPDF_Page_LineTo(active_page, pwidth - MARGIN_RIGHT, box_it - merged_box_height);
            HPDF_Page_Stroke(active_page);
            if (boxes_left == total_boxes) {
                HPDF_Page_MoveTo(active_page, MARGIN_LEFT, box_it);
                HPDF_Page_LineTo(active_page, pwidth - MARGIN_RIGHT, box_it);
                HPDF_Page_Stroke(active_page);
            }
            if (boxes_left == drawable_boxes) {
                HPDF_Page_MoveTo(active_page, MARGIN_LEFT, box_it - merged_box_height);
                HPDF_Page_LineTo(active_page, pwidth - MARGIN_RIGHT, box_it - merged_box_height);
                HPDF_Page_Stroke(active_page);
            }
```

```
            // deduced drawn boxes
            box_it -= merged_box_height;
            last_boxed_page = active_page;
            LOG("merged %d boxes, total height : %.2f", drawable_boxes, merged_box_height);
        }

        const char *line = lines[i];
        bool is_comment = ww_is_comment(line);
        HPDF_Font font = is_comment ? c->font_codecomment : c->font_codetext;
        struct vec3 txt_clr = is_comment ? c->clr_codecomment : c->clr_codetext;
        float font_height = code_font_height; // ww_get_curfont_height(active_page);

        HPDF_Page_SetRGBFill(active_page, txt_clr.x, txt_clr.y, txt_clr.z);
        HPDF_Page_SetRGBStroke(active_page, txt_clr.x, txt_clr.y, txt_clr.z);
        HPDF_Page_SetFontAndSize(ww_get_active_page(c), font, 8.0);
        ww_textout(c, line);
    }

    fclose(file);
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_pushback_a4page(struct config *c)
{
    HPDF_Page page = HPDF_AddPage(c->pdf);
    HPDF_Page_SetSize(page, HPDF_PAGE_SIZE_A4, HPDF_PAGE_PORTRAIT);
    c->book.push_back(page);
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_add_heading(struct config *c, const char *h)
{
    HPDF_Page_SetFontAndSize(ww_get_active_page(c), c->font_heading, 12.0);
    ww_textout(c, h);
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_add_exec_file(struct config *c, const char *f)
{
    ww_textout(c, "output goes here");
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_addlinebg(HPDF_Page page, struct vec3 clr, float x, float y, float width, float height)
{
    HPDF_Page_SetRGBFill(page, clr.x, clr.y, clr.z);
    HPDF_Page_SetRGBStroke(page, clr.x, clr.y, clr.z);
    HPDF_Page_Rectangle(page, x, y, width, height);
    HPDF_Page_Fill(page);
    return OK;
}
```

```
////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_textout(struct config *c, const char *msg)
{
    // The new line is placed, treating the c->it (iterator) as the top-left corner
    // of the line.

    HPDF_Page active_page = ww_get_active_page(c);
    HPDF_Font active_font = HPDF_Page_GetCurrentFont(active_page);
    float font_height = ww_get_curfont_height(active_page);
    float page_height = HPDF_Page_GetHeight(active_page);

    // iterator too high on the page?
    if (c->it > page_height - MARGIN_TOP)
        c->it = page_height - MARGIN_TOP;


    // NOTE : We can always write at c->it - font_height (this line) because when
    // c->it - font_height * 2.0 (next line) goes out of bound, we push_back new page.
    HPDF_Page_BeginText(active_page);
    HPDF_Page_TextOut(active_page, MARGIN_LEFT, c->it - font_height, msg);
    HPDF_Page_EndText(active_page);
    c->it -= font_height;


    // Next line out of range? pushback new page
    if (c->it - font_height < MARGIN_BOTTOM) {
        ww_pushback_a4page(c);

        HPDF_Page new_page = ww_get_active_page(c);
        ww_copy_page_stats(active_page, new_page);

        active_page = new_page;
        c->it = page_height - MARGIN_TOP;
    }

    return OK;
}


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_copy_page_stats(HPDF_Page from, HPDF_Page to)
{
    // font & size
    HPDF_Page_SetFontAndSize(to, HPDF_Page_GetCurrentFont(from), HPDF_Page_GetCurrentFontSize(from));

    // fill clr
    HPDF_RGBColor from_clr = HPDF_Page_GetRGBFill(from);
    HPDF_Page_SetRGBFill(to, from_clr.r, from_clr.g, from_clr.b);

    // stroke clr
    HPDF_RGBColor from_stroke_clr = HPDF_Page_GetRGBStroke(from);
    HPDF_Page_SetRGBStroke(to, from_stroke_clr.r, from_stroke_clr.g, from_stroke_clr.b);
    return OK;
}


////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////
static int ww_split_line(HPDF_Page page, char *line, std::vector<char*>* lines_out)
{
    float pwidth = HPDF_Page_GetWidth(page);
```

```
    float split_width = pwidth - MARGIN_RIGHT - MARGIN_LEFT;

    while (*line != '\0') {
        HPDF_REAL real_width = 0.0;
        size_t split_len = HPDF_Page_MeasureText(page, line, split_width, true, &real_width);
        char *line_buf = (char*)calloc(split_len + 2, 1);
        strncpy(line_buf, line, split_len);
        lines_out->push_back(line_buf);
        line += split_len;
    }
    return OK;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static int ww_is_comment(const char *c)
{
    size_t i = 0;
    while (c[i] != '\0') {
        if (c[i] != ' ')
            return strncmp(&c[i], COMMENT_STR, sizeof(COMMENT_STR) - 1) == 0;
        ++i;
    }

    return 0;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
void err_handler(HPDF_STATUS error_no, HPDF_STATUS detail_no, void *user_data)
{
    FAIL_LOG("libharu error [ %d ], detail no %d\n", (int)error_no, (int)detail_no);
    exit(1);
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
HPDF_Font init_font(HPDF_Doc pdf, const char *font_file)
{
    const char *font_name = HPDF_LoadTTFontFromFile(pdf, font_file, HPDF_TRUE);
    if (!font_name) {
        FAIL_LOG("Failed to load font from file");
        return NULL;
    }

    return HPDF_GetFont(pdf, font_name, NULL);
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static float ww_get_curfont_height(HPDF_Page page)
{
    HPDF_Font f = HPDF_Page_GetCurrentFont(page);
    float height = (HPDF_Font_GetAscent(f) - HPDF_Font_GetDescent(f)) * HPDF_Page_GetCurrentFontSize(page) / 1000.0
;

    return height * 1.2;
}
```

```
////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
static HPDF_Page ww_get_active_page(struct config *c)
{
    return c->book.back();
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
HPDF_Page add_a4_page(HPDF_Doc* pdf)
{
    HPDF_Page page = HPDF_AddPage(*pdf);
    HPDF_Page_SetSize(page, HPDF_PAGE_SIZE_A4, HPDF_PAGE_PORTRAIT);
    return page;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
size_t linelen(const char *s)
{
    size_t l = 0;
    while (s[l] != '\0' && s[l] != '\n')
        ++l;

    return l;
}


////////////////////////////////////////////////////////////////////////////
////////////////////////////////////////////////////////////////////////////
void sanitize_strline_inplace(char *str)
{
    size_t tab_size = 4;
    size_t shift_size = tab_size - 1;
    size_t i = 0;
    int tabs = 0;
    while (str[i] != '\0' && str[i] != '\n') {
        if (str[i] == '\t') {
            for (size_t j = linelen(str); j > i; --j)
                str[j + shift_size] = str[j];
            for (size_t j = i; j < i + tab_size; ++j)
                str[j] = ' ';
        }
        ++i;
    }

    str[i] = '\0';
}
```

OUTPUT

OUTPUT GOES HERE